

Machine-Learning-Driven Intelligent Memory System Design: From On-Chip Caches to Storage

Rahul Bera¹, Rakesh Nadig², and Onur Mutlu³, ETH Zürich, Zürich, 8092, Switzerland

Despite the data-rich environment in which memory systems of modern computing platforms operate, many state-of-the-art architectural policies employed in the memory system rely on static, human-designed heuristics that fail to truly adapt to the workload and system behavior via principled learning methodologies. In this article, we propose a fundamentally different design approach: using lightweight and practical machine learning (ML) methods to enable adaptive, data-driven control throughout the memory hierarchy. We present three ML-guided architectural policies: 1) Pythia, a reinforcement learning (RL)-based data prefetcher for on-chip caches; 2) Hermes, a perceptron learning-based off-chip predictor for multilevel cache hierarchies; and 3) Sibyl, an RL-based data placement policy for hybrid storage systems. Collectively, this article demonstrates that integrating adaptive learning into memory subsystems can lead to intelligent, self-optimizing architectures that unlock performance and efficiency gains beyond what is possible with traditional human-designed approaches.

Today's computation is heavily bottlenecked by data. Many key applications [e.g., machine learning (ML), graph analytics, large-scale recommendation systems, and genome sequencing and analysis], irrespective of whether they are executed in cloud servers or mobile devices, are all data intensive.^{1,2,3,9,10,12,13,14} They operate on large numbers of application data that overwhelm the storage and retrieval capability of high-performance memory systems employed in today's processors, rendering memory as the key performance and energy bottleneck.⁴ To alleviate this, architects have progressively optimized the memory system with numerous sophisticated speculative (e.g., prefetching data to on-chip caches before the processor demands it)⁵ and control (e.g.,

placing a data block in an appropriate device in a hybrid memory/storage system)⁶ policies to reduce the data access overheads.

Even though these architectural policies observe vast numbers of application data (as well as system-generated metadata) during their online operation, we observe that these policies often make simple decisions based on simple, rigid heuristics, unable to learn in a principled manner from massive numbers of easily available data. This is primarily because most existing architectural policies make *human-design-driven* decisions, which predominantly rely on fixed and often myopic human-crafted heuristics that provide limited adaptability to complex system state and workload demands. For example, architects have proposed numerous data prefetching policies to predict the addresses of future memory requests and fetch their data into traditional on-chip caches before the processor demands it. Yet most prefetching policies rely on a single piece of program context information (e.g., the program counter value of a load instruction), that a human architect selects at design time, to identify

0272-1732 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.

Digital Object Identifier 10.1109/MM.2026.3667076

Date of publication 26 February 2026; date of current version 17 June 2026.

patterns in a program's memory access stream. As a result, these prefetching policies often fail to adapt to changing workload behavior across a diverse set of workloads.⁷

Our goal is to improve upon many such rigid human-designed policies found in modern state-of-the-art memory system designs by making them *data driven* and fundamentally adaptive in a principled manner. To this end, we propose harnessing lightweight and practical ML to guide architectural decision making, enabling intelligent, data-driven control policies throughout the memory hierarchy: from the on-chip caches to the storage subsystem.

Specifically, we describe three ML-driven architectural policies: 1) *Pythia*,⁷ a reinforcement learning (RL)-based framework for intelligent hardware prefetching in on-chip caches; 2) *Hermes*,⁸ a perceptron learning-based mechanism for accelerating long-latency memory requests via accurate off-chip load prediction in a multilevel cache hierarchy; and 3) *Sibyl*,⁶ an RL-based adaptive data placement policy for hybrid storage systems.

Our evaluation shows that Pythia, Hermes, and Sibyl not only provide significant performance and energy improvements on top of a well-optimized memory hierarchy design but also consistently outperform prior best human-designed policies across a wide range of workloads and system configurations due to their adaptive and principled data-driven decision making. With detailed hardware design and prototyping, we also show that all three proposed policies not only incur a modest area and power overheads but can be practically employed in today's computing systems.

Collectively, these proposals illustrate the significant potential of ML-driven policies to design adaptive and self-optimizing memory systems, fundamentally transforming architectural decision-making processes.

We make the following key contributions in this article:

- We analyze various human-designed state-of-the-art architectural policies proposed for three components in the memory system: data prefetching, off-chip prediction, and data placement. We show that these policies leave a significant performance potential on the table due to their rigid and myopic designs.
- We propose three novel policies, Pythia, Hermes, and Sibyl, that employ various forms of ML (reinforcement and perceptron learning) (see “[A Primer on Reinforcement Learning and Perceptron Learning](#)”) to make adaptive and principled data-driven decisions online.

- We show that Pythia, Hermes, and Sibyl provide significant benefits over the best human-designed policies across a wide range of workloads and system configurations.

SHORTCOMINGS OF HUMAN-DESIGNED ARCHITECTURAL POLICIES

The ever-growing data footprint of many modern workloads (and likely future workloads) greatly overwhelms the storage and retrieval capability of the memory systems of modern machines. As such, data access has become the key performance and energy bottleneck. In response, architects have developed numerous speculative (e.g., data prefetching) and control (e.g., data placement and management) policies to mitigate data access overheads. Although these policies observe a vast quantity of application data and system-generated metadata during their online operation, their decision-making process often relies on simple and often myopic human-designed heuristics, failing to dynamically adapt to complex system states and evolving workload demands in a principled manner. As a result, they often leave a large potential for performance and energy-efficiency improvement on the table.

In this section, we motivate the need for *data-driven* policies by examining the shortcomings of conventional human-design-driven policies through three case studies that span the memory hierarchy: from the on-chip caches to the storage system.

Case Study 1: Prefetching in On-Chip Caches

Prefetching is a well-studied speculation technique that predicts the addresses of long-latency memory requests and fetches their corresponding data from the main memory to on-chip caches before the processor demands it. To identify patterns within a program's memory access stream, a prefetcher typically correlates memory addresses with program context information (also called *program feature*).

Numerous prefetching techniques proposed in the literature consistently exhibit two primary limitations: 1) reliance on a *single, static, human-designed* program feature to identify memory access patterns, significantly limiting their efficacy across diverse workloads, and 2) lack of *inherent system awareness* (e.g., memory bandwidth usage), resulting in performance degradation in resource-constrained systems.

To illustrate this, we show the coverage (i.e., the fraction of program memory requests correctly predicted

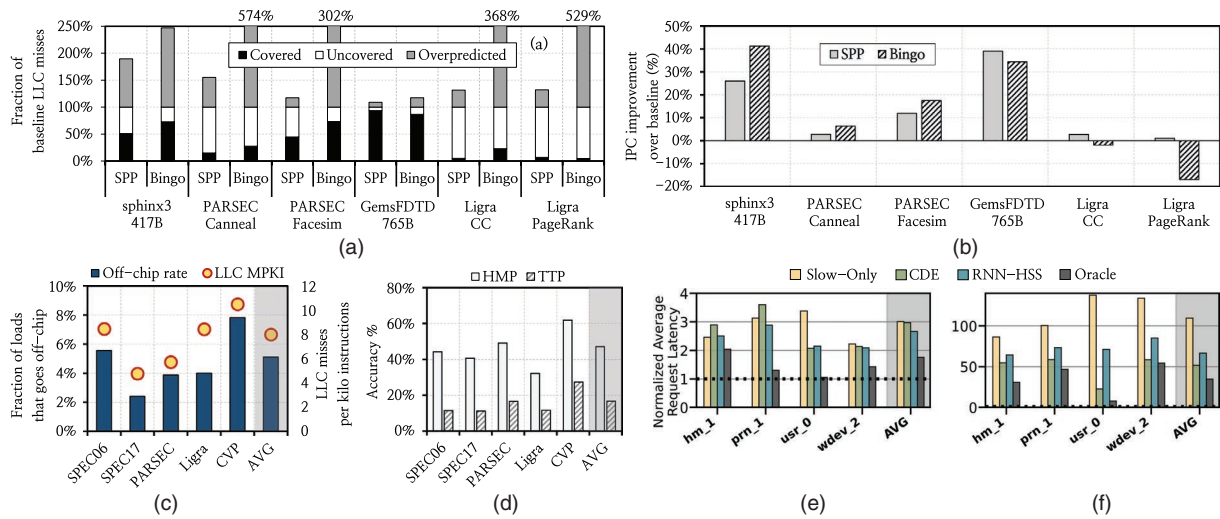


FIGURE 1. (a) Coverage and overprediction and (b) performance comparison of two recently proposed prefetchers: SPP and Bingo. (c) Percentage of loads that miss the LLC and go off-chip (on the left y-axis) and the LLC misses per kilo instruction (on the right y-axis) in the baseline system with a state-of-the-art prefetcher. (d) Accuracy of two off-chip predictors: HMP and TTP. Average request latency of CDE and RNN-HSS on (e) performance-oriented HSS and (f) cost-oriented HSS. The average request latency is normalized to Fast-Only policy. LLC: last-level cache; HMP: hit-miss predictor; TTP: tag-tracking-based predictor; CDE: cold data eviction; RNN-HSS recurrent neural network-hybrid storage system. Figures adapted from our MICRO 2021,⁷ MICRO 2022,⁸ and ISCA 2022⁶ papers.

by the prefetcher) and overpredictions (i.e., prefetched memory requests that are not demanded by the program) of two recently proposed prefetchers, signature path prefetcher¹² (SPP) and Bingo,¹³ for six example workloads in Figure 1(a) and their performance improvement in Figure 1(b). We make two key observations from this figure. First, as SPP and Bingo rely on two different yet static program features, neither provides the best performance benefit even across this small set of workloads. Second, even if Bingo achieves a similar prefetch coverage in Ligra-CC as compared to PARSEC-Canneal while generating significantly lower overpredictions, Bingo *degrades* performance by 1.9% in Ligra-CC but *improves* performance by 6.4% in PARSEC-Canneal, over a no-prefetching baseline. This contrasting outcome is due to Bingo's lack of awareness of memory bandwidth usage of the system.

We conclude that, despite extensive research in prefetching, the static and human-designed nature of most conventional prefetching policies significantly limits their efficacy over a wide range of workloads and system configurations.

Case Study 2: Off-Chip Prediction in Multilevel Cache Hierarchies

To cater to the ever-increasing data footprints of modern workloads, architects continue to increase the size

(and also number of levels) of the on-chip caches in commercial processors, which inadvertently increases on-chip cache access latencies. As a result, a large fraction of the latency of an off-chip memory request is spent accessing the on-chip cache hierarchy to solely determine that it needs to go off-chip.

To mitigate this, architects have proposed *off-chip prediction*, a technique that predicts which memory requests might go off-chip and speculatively fetches its data *directly* from the main memory.⁸ However, predicting which memory requests will miss the entire cache hierarchy is challenging, especially due to the presence of complementary speculative techniques like data prefetching. Our analysis, as depicted in Figure 1(c), shows that on average, only 5.1% of the total loads generated by a workload goes off-chip in presence of a sophisticated data prefetcher. Although researchers have proposed techniques to accurately predict off-chip memory requests, these techniques either rely on a fixed set of human-crafted program features to correlate with the off-chip requests or expensive hardware structures to track cacheline tags in the entire cache hierarchy. Yet they fall short in providing accurate off-chip prediction across a wide range of workloads. As Figure 1(d) shows, the prior-best hit-miss predictor (HMP) and the tag-tracking-based predictor (TTP) provide only 47% and 16.6%

accuracy (i.e. the fraction of predicted off-chip loads that actually go off-chip), respectively.

This shows that the current off-chip predictors have a significant room for improvement due to their reliance on fixed, human-crafted program features.

Case Study 3: Data Placement in Hybrid Storage Systems

This case study explores the storage system, another critical component of the memory hierarchy. Modern high-performance systems employ hybrid storage, combining fast-yet-small storage devices (e.g., solid-state devices) with large-yet-slow devices (e.g., hard disk drive) to deliver high capacity at low latency. The key challenge in designing a high-performance, cost-effective hybrid storage system (HSS) is to accurately identify the performance-critical application data and place it in the best-fit storage device.

Prior data placement techniques propose handcrafted policies that 1) consider only a limited number of workload characteristics (e.g., access frequency or access recency) to identify the storage device for an incoming input-output (I/O) request and 2) do not consider the variations in read/write latencies of storage devices, or the number and types of storage devices in the HSS. As a result, such data placement techniques cannot easily adapt to the dynamic workload demands and diverse real-world hybrid storage system configurations. To demonstrate this lack of adaptivity, we show the average request latencies of two prior data-placement techniques, cold data eviction (CDE) and RNN (recurrent neural network)-HSS, for four representative workloads on a performance-oriented HSS in Figure 1(e) and cost-oriented HSS in Figure 1(f). We add three boundary scenarios to our evaluation: 1) Slow-Only, where

all the data are placed in the slow storage device; 2) Fast-Only, where all the data are placed in the fast storage device; and 3) Oracle, which performs data placement with complete knowledge of future I/O access patterns.

We make two key observations. First, CDE and RNN-HSS show a large average performance loss compared to Oracle: 41.1% (32.6%) and 34.4% (47.6%) in performance-oriented (cost-oriented) HSS, respectively. Although both techniques achieve comparable performance to Oracle in select workloads (e.g., `usr_0` for CDE in cost-oriented HSS, `hm_1` for RNN-HSS in performance-oriented HSS), they show suboptimal performance in other workloads, showing limited adaptability to workload diversity. Second, both techniques demonstrate inconsistent behavior across the two HSS configurations: in `hm_1` workload, the both CDE and RNN-HSS underperform the Slow-Only in performance-oriented HSS but outperform the Slow-Only in cost-oriented HSS. This shows the inability of prior approaches to adapt to varying storage device characteristics and system heterogeneity.

We conclude that prior handcrafted data placement techniques fail to adapt to diverse workload demands or changes in storage device characteristics due to their rigid design choices and limited system awareness.

DATA PREFETCHING USING RL

Formulating Prefetching as an RL Agent

We formulate prefetching as an RL problem, as shown in Figure 2(a). Specifically, Pythia⁷ is an RL agent that learns accurate, timely, and system-aware prefetch decisions by interacting with the processor and memory

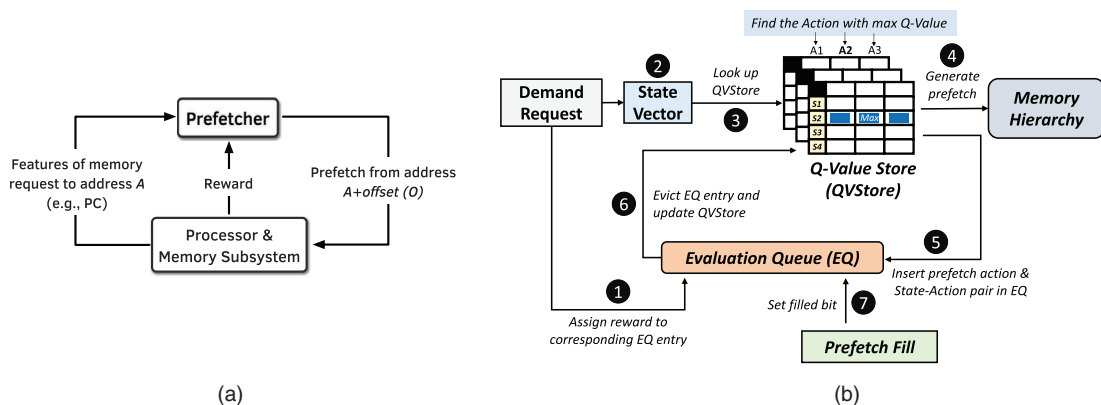


FIGURE 2. (a) Formulating prefetcher as an RL agent. (b) Overview of Pythia. Q value: quality value. EQ: evaluation queue; max: maximum. Figures adapted from our MICRO 2021.⁷

A Primer on Reinforcement Learning and Perceptron Learning

WHAT IS REINFORCEMENT LEARNING?

Reinforcement learning (RL)¹ is the algorithmic approach to learn how to take an *action* in a given *situation*. A typical RL system consists of an agent and an environment. At every time step t , the agent observes the current *state* of the environment S_t and takes an *action* A_t , for which the environment provides a *reward* R_{t+1} . The agent's goal is to find the optimal *policy* that maximizes the cumulative reward collected from the environment.

The expected cumulative reward by taking an action A in a given state S is defined as the *quality* (Q) *value* of the state-action pair [denoted as $Q(S,A)$]. The agent iteratively optimizes its policy in two steps: 1) updating Q values using the reward collected from the environment and 2) optimizing the policy using the updated Q value. This form of RL is called *Q-learning*.¹

Prior works have explored the use of RL in designing memory controllers² and software-aware prefetchers.³ In this article, we demonstrate Pythia, the first practical, software-transparent data prefetcher using RL.

WHAT IS PERCEPTRON LEARNING?

Perceptron learning is a simplified learning model that mimics biological neurons.⁴ In its simplistic form, a *single-layer perceptron* network is composed of multiple numerical inputs and one output. Each input is connected to the output via an *artificial neuron*, which is represented by a numeric value, called *weight*. The perceptron network as a whole iteratively learns a binary classification function $f(x)$ [shown in (S1)], a function that maps the input X (a vector of n values) to a single binary output.

$$f(x) = \begin{cases} 1 & \text{if } w_0 + \sum_{i=1}^n w_i x_i > 0 \\ 0 & \text{otherwise.} \end{cases} \quad (\text{S1})$$

The perceptron learning algorithm starts by initializing the weight of each neuron and iteratively trains the weights by calculating the error between

the output computed using each input vector from the training dataset and the desired output.

Prior research has successfully demonstrated perceptron learning for predicting branch outcomes,⁵ cache reuse,⁶ and prefetch usefulness.⁷ In this article, we describe Hermes, which is the first work that applies perceptron learning for off-chip prediction.

REFERENCES

1. R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
2. E. Ipek, O. Mutlu, J. F. Martínez, and R. Caruana, "Self-optimizing memory controllers: A reinforcement learning approach," in *Proc. 35th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2008, pp. 39–50.
3. L. Peled, S. Mannor, U. Weiser, and Y. Etsion, "Semantic locality and context-based prefetching using reinforcement learning," in *Proc. ACM/IEEE 42nd Annu. Int. Symp. Comput. Archit. (ISCA)*, 2015, pp. 285–297, doi: [10.1145/2749469.2749473](https://doi.org/10.1145/2749469.2749473).
4. F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Rev.*, vol. 65, no. 6, pp. 386–408, 1958, doi: [10.1037/h0042519](https://doi.org/10.1037/h0042519).
5. D. A. Jiménez and C. Lin, "Dynamic branch prediction with perceptrons," in *Proc. HPCA 7th Int. Symp. High-Perform. Comput. Archit.*, 2001, pp. 197–206, doi: [10.1109/HPCA.2001.903263](https://doi.org/10.1109/HPCA.2001.903263).
6. E. Teran, Z. Wang, and D. A. Jiménez, "Perceptron learning for reuse prediction," in *Proc. 49th Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2016, pp. 1–12, doi: [10.1109/MICRO.2016.7783705](https://doi.org/10.1109/MICRO.2016.7783705).
7. E. Bhatia, G. Chacon, S. Pugsley, E. Teran, P. V. Gratz, and D. A. Jiménez, "Perceptron-based prefetch filtering," in *Proc. ACM/IEEE 46th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2019, pp. 1–13.

subsystem. At each time step, corresponding to a new demand request, Pythia observes system state and takes a prefetch action. Each action (including not issuing a prefetch) yields a numerical reward based on the accuracy and timeliness of the prefetch request, and various system-level feedback information. Pythia's goal is to identify an optimal policy maximizing accurate and timely prefetches, while taking system-level feedback information into account. Although Pythia's framework can be extended to various system-level feedback types, in this work, we demonstrate Pythia using memory bandwidth usage as a feedback.

State

We define the state as a k -dimensional vector of program features, each comprising two components: 1) control flow and 2) data flow. The control-flow component includes basic information like load-PC or branch-PC, and a history indicating whether it is derived from current or past requests. The data-flow component includes cacheline address, physical page number, page offset, cacheline delta, and associated history. Although Pythia can utilize many features, we fix the state-vector size (i.e., k) at design time given a limited hardware storage budget. However, the exact set of k features is configurable online via configuration registers.

Action

We define the action of the RL agent as selecting a prefetch offset (i.e., a delta between the predicted and demanded cacheline addresses) from a set of candidate prefetch offsets. To limit prefetch requests within the physical page of the triggering demand request, the list of prefetch offsets contains only values in the range of $[-63, 63]$ for a system with a traditionally sized 4-KB page and 64B cacheline. A zero offset, which signifies that no prefetch request is generated, is also a valid action for Pythia.

Reward

We define five different reward levels:

1. *Accurate and timely* ($\mathcal{R}_{AT}$), assigned to an action whose corresponding prefetch address gets demanded *after* the prefetch fill.
2. *Accurate but late* ($\mathcal{R}_{AL}$), assigned to an action whose corresponding prefetch address gets demanded *before* the prefetch fill.
3. *Loss of coverage* ($\mathcal{R}_{CL}$), assigned to an action whose corresponding prefetch address is to a different physical page than the triggering demand request.

4. *Inaccurate* ($\mathcal{R}_{IN}$), assigned to an action whose corresponding prefetch address does *not* get demanded in a temporal window. The reward is classified into two sublevels: inaccurate given low bandwidth usage ($\mathcal{R}_{IN}^L$) and inaccurate given high bandwidth usage ($\mathcal{R}_{IN}^H$).
5. *No-prefetch* ($\mathcal{R}_{NP}$), assigned when Pythia decides not to prefetch. This reward level is also classified into two sublevels: no-prefetch given low bandwidth usage ($\mathcal{R}_{NP}^L$) and no-prefetch given high bandwidth usage ($\mathcal{R}_{NP}^H$).

The reward levels, in unison, provide prefetching objectives to Pythia. $\mathcal{R}_{AT}$ and $\mathcal{R}_{AL}$ guide Pythia toward accurate and timely prefetches. $\mathcal{R}_{CL}$ encourages prefetching within the same physical page as the triggering request. $\mathcal{R}_{IN}$ and $\mathcal{R}_{NP}$ shape Pythia's strategy based on memory bandwidth usage.

Pythia: Design Overview

Figure 2(b) shows a high-level overview of Pythia, comprising two primary hardware structures: *Q-Value Store* (QVStore) and *Evaluation Queue* (EQ). QVStore records Q values for observed state-action pairs. EQ maintains a first-in, first-out list of recently taken actions, where each entry contains three information: 1) the selected action, 2) corresponding prefetch address, and 3) a *filled* bit indicating cache fill status.

Upon receiving a demand request, Pythia checks the EQ for the requested memory address (❶). If present, it implies that a previously issued prefetch was useful, prompting Pythia to assign a reward (either $\mathcal{R}_{AT}$ or $\mathcal{R}_{AL}$) depending on the filled bit. Next, Pythia constructs a state vector from the demand request attributes (e.g., PC, address, or cacheline delta) (❷) and consults QVStore to identify the action with the highest Q value (❸). Pythia selects this action and issues the corresponding prefetch request to the cache hierarchy. Simultaneously, Pythia inserts the chosen action, its prefetch address, and state vector into EQ (❹). Actions involving no prefetch or addresses outside the current physical page are also inserted, with rewards assigned immediately. Upon eviction of an EQ entry, Pythia updates Q values using the stored state-action pair and reward (❺). When a prefetch request gets filled in cache, Pythia sets the filled bit in its EQ entry (❻), marking it as timely or late.

Storage Overhead

Overall, Pythia requires only 25.5-KB storage per core, in which QVStore and EQ consume 24 and 1.5 KB, respectively.⁷

Evaluation Methodology

We use the ChampSim trace-driven simulator to evaluate Pythia. We simulate an Intel Skylake-like multicore processor that supports up to 12 cores. For single-core (multicore) simulations, we warm up the core using 100 million (50 million) instructions, followed by simulating the next 500 million (150 million) instructions. We evaluate Pythia using a diverse set of 150 memory-intensive workload traces spanning SPEC CPU2006, SPEC CPU2017, PARSEC, Ligr, and Cloudsuite benchmark suites and compare it against five state-of-the-art prior prefetchers: SPP, SPP+PPF, SPP+DSPatch, Bingo, and multi-lookahead offset prefetching (MLOP). All prefetchers are trained on (level-one) L1-cache misses and fills prefetched lines into L2 and last-level cache (LLC). We also prototype Pythia using Chisel HDL to accurately estimate its area, power, and latency overhead, and incorporate them in the performance model.⁷

Key Results

Performance Evaluation With Varying Number of Cores

Figure 3(a) illustrates the average performance improvement across all traces for prefetchers in single- to 12-core systems. We make two key observations. First, Pythia consistently outperforms MLOP, Bingo, and SPP across *all* configurations. Second, Pythia's performance gain over all evaluated prior prefetchers increases with an increase in the number of processor cores. As core count increases, per-core memory bandwidth becomes increasingly constrained, limiting the effectiveness of bandwidth-unaware prefetchers. By explicitly adapting its prefetching policy based on bandwidth usage, Pythia achieves larger performance gains as core count increases. In single-core systems, Pythia outperforms MLOP, Bingo, SPP, and aggressive

SPP with perceptron filtering (PPF) by 3.4%, 3.8%, 4.3%, and 1.02%, respectively. For four-core (and 12-core) systems, Pythia exceeds MLOP, Bingo, SPP, and SPP+PPF by 5.8% (7.7%), 8.2% (9.6%), 6.5% (6.9%), and 3.1% (5.2%), respectively.

Performance Evaluation With Varying Dynamic Random-Access Memory Bandwidth

To evaluate Pythia for bandwidth-constrained commercial server-class processors [where each core receives only a fraction of a dynamic random-access memory (DRAM) channel], we simulate a single-core, single-channel configuration by scaling DRAM bandwidth [Figure 3(b)]. Pythia consistently outperforms all competing prefetchers across bandwidth levels ranging from $1/16 \times$ to $4 \times$ the baseline. MLOP and Bingo suffer substantial performance drops under low-DRAM bandwidth due to overprediction. By trading prefetch coverage for accuracy based on bandwidth usage, Pythia outperforms MLOP, Bingo, SPP, and SPP+PPF by 16.9%, 20.2%, 3.7%, and 9.5%, respectively, in the most constrained 150-MTPS (million transfers per second) configuration. Even under ample bandwidth at 9600 MTPS, Pythia leads MLOP, Bingo, SPP, and SPP+PPF by 3%, 2.7%, 4.4%, and 0.8%, respectively.

These results succinctly demonstrate that Pythia's RL-based prefetching policy dynamically adapts to a wide range of workloads and system configurations, providing *consistent* performance gains over many prior-best human-designed prefetchers. More results can be found in our MICRO 2021 paper.⁷

OFF-CHIP PREDICTION VIA PERCEPTRON LEARNING

Hermes: Design Overview

Figure 4(a) provides a high-level overview of Hermes.⁸ perceptron-based off-chip load predictor (POPET) is a key component of Hermes that accurately predicts which load requests will go off-chip (i.e., access DRAM) by correlating a load request with multiple program features via perceptron learning. For each processor-generated demand load instruction, POPET predicts whether the request will go off-chip (❶). If so, Hermes issues a speculative memory request (called a *Hermes request*) directly to the main memory controller immediately after obtaining the physical address (❷). This Hermes request proceeds in parallel with the regular load accessing the on-chip cache. If the prediction is correct, the regular load eventually misses in the LLC and waits for the ongoing Hermes request,

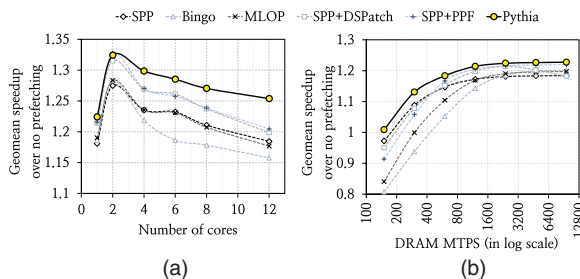


FIGURE 3. Average performance improvement of prior prefetchers and Pythia in systems with (a) a varying number of cores and (b) dynamic random-access memory (DRAM) million transfers per second (MTPS). Figures adapted from our MICRO 2021 paper.⁷

effectively hiding on-chip cache access latency (③). If the Hermes request completes but no regular load targets the same address (e.g., a predicted off-chip load request hitting on-chip cache), Hermes discards it, avoiding unnecessary cache fills and preserving coherence. Hermes uses each retired load to train POPET for future predictions (④).

Design of the POPET Off-Chip Load Predictor

POPET is composed of multiple 1-D weight tables, each associated with a single program feature. Each table entry holds a 5-bit saturating signed integer that records the correlation between a feature value and the actual outcome (i.e., whether the load actually went off-chip). A weight value closer to +15 or -16 indicates strong positive or negative correlation, respectively, while values near zero indicate weak correlation. During the training [step ④ in Figure 4(a)], these weights are updated based on the true outcome to refine POPET's predictions.

Making Predictions Using POPET

During load queue (LQ) allocation for a core-generated load [step ① in Figure 4(a)], POPET performs a binary prediction on whether the load will go off-chip, following a three-stage process shown in Figure 4(b). First, POPET extracts various program features from the current load and prior requests. Second, it hashes each feature value to index into the corresponding weight table and retrieves the weight. Third, it sums all weights to compute the cumulative perceptron weight (W_σ). If W_σ exceeds the activation threshold (τ_{act}), POPET predicts that the load will go off-chip. The hashed feature values, W_σ , and the prediction are

stored in the LQ entry to train POPET when the load completes [step ④ in Figure 4(a)].

Training POPET

POPET training begins when a demand load returns to the core and releases its LQ entry [step ④ in Figure 4(a)]. Loads that miss the LLC and access main memory are labeled true off-chip requests. POPET uses this true outcome and the stored prediction in the LQ entry to update feature weights in two stages. First, the previously computed W_σ is retrieved. If W_σ lies between the positive and negative training thresholds, T_P and T_N , respectively, training is triggered. This check prevents oversaturation of weights and enables quick adaptation to program phase changes. Second, if training proceeds, POPET retrieves each feature's weight using the hashed indices from the LQ entry. If the true outcome is positive (off-chip), the weights are incremented by one; if negative, they are decremented by one. This update mechanism steers each feature weight toward the true outcome, improving prediction accuracy over time.

Storage Overhead

Overall, Hermes requires only 4 KB of storage per core, in which POPET and LQ metadata consume 3.2 and 0.8 KB, respectively.⁸

Evaluation Methodology

We use the ChampSim trace-driven simulator to evaluate Hermes. We faithfully model the latest-generation Intel Alder Lake performance core with its large reorder buffer and large caches with publicly reported on-chip cache access latencies. For single-core (multicore) simulations, we warm up the core using 100

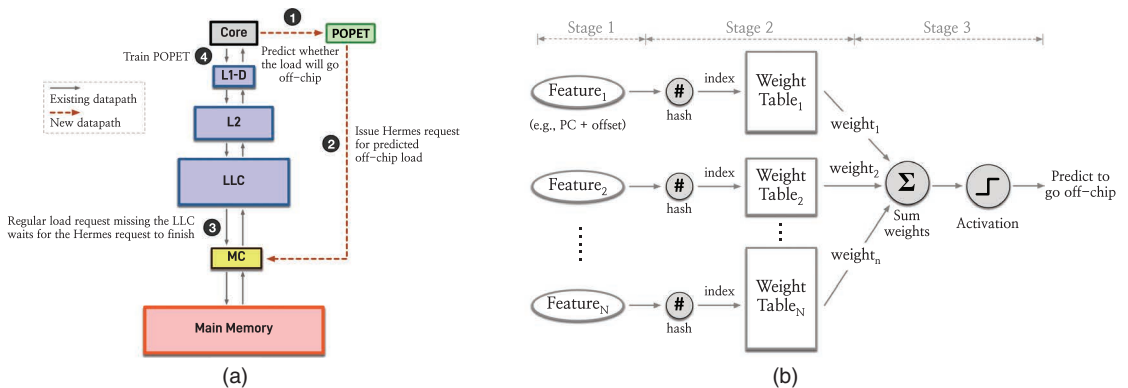


FIGURE 4. (a) Overview of Hermes. (b) Hardware stages to make a prediction by POPET. MC: memory controller. Figures adapted from our MICRO 2022 paper.⁸

million (50 million) instructions, followed by simulating the next 500 million (100 million) instructions. We evaluate Hermes using 110 memory-intensive workload traces spanning SPEC CPU2006, SPEC CPU2017, PARSEC, LiGra, and commercial workloads from the 2nd data value prediction championship (CVP), and compare Hermes against 1) five state-of-the-art data prefetchers (i.e., Pythia,⁷ Bingo, SPP with PPF, MLOP, and SMS) and 2) two prior off-chip prediction mechanisms: hit/miss predictor (HMP) and cacheline tag-tracking predictor (TTP). We also evaluate Hermes using a range of latencies for issuing a Hermes request directly to the memory controller [step ② in Figure 4(a)] to account for diverse on-chip interconnect designs.⁸

Key Results

Performance Evaluation With Different Off-Chip Predictors

Figure 5(a) shows the performance of Hermes with POPET, HMP, TTP, and an ideal off-chip predictor, with and without an underlying baseline prefetcher (in this case, Pythia) and normalized to a no-prefetching system for single-core workloads. We highlight two key observations. First, Hermes with POPET outperforms Hermes-HMP and Hermes-TTP. This is due to POPET's high accuracy off-chip prediction (77.1% on average) as compared to HMP (47%) and TTP (16.6%).^a On average, Hermes-HMP, Hermes-TTP, and Hermes with POPET improve performance over Pythia by 0.8%, 1.7%, and 5.4%, respectively. Second, Hermes-POPET achieves nearly 90% of the performance gain of the ideal off-chip predictor that assumes perfect prediction. These results show that POPET's high prediction accuracy

^a Our MICRO 2022 article further shows that POPET's high-accuracy off-chip prediction incurs minimal overhead: increasing main memory requests by 5.5% and dynamic power by 3.6% over no prefetching.⁸

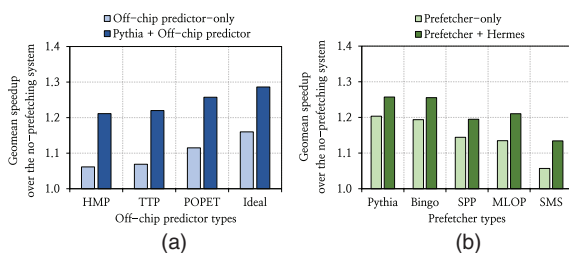


FIGURE 5. Performance comparison by varying (a) off-chip predictors and (b) the underlying prefetcher. Figures adapted from our MICRO 2022 paper.⁸

and coverage are central to Hermes's effectiveness, underscoring the importance of a well-designed off-chip predictor.

Performance Evaluation With Different Underlying Prefetchers

We evaluate Hermes naïvely combined (i.e., without any implicit coordination) with five state-of-the-art data prefetchers at LLC. Figure 5(b) shows the performance normalized to a no-prefetching single-core baseline. Hermes combined with any prefetcher *consistently* outperforms the prefetcher alone. Specifically, Hermes improves performance over Bingo, SPP, MLOP, and SMS by 6.2%, 5.1%, 7.6%, and 7.7%, respectively. Our recent work, Athena,¹¹ shows that an RL-based coordination policy to synergize off-chip predictor with prefetcher can provide even more performance improvement than their naïve combination.

These results suggest that by employing a learning-based, adaptive off-chip prediction mechanism, Hermes *consistently* outperforms prior off-chip predictors across diverse workloads and system configurations. More results can be found in our 2022 *IEEE Micro* article.⁸

DATA PLACEMENT USING RL

We propose Sibyl,⁶ an RL-based data placement technique for hybrid storage systems. Although RL is a promising alternative to existing data placement techniques, its effectiveness depends on how the data placement problem is cast as an RL task.

Formulating Data Placement as an RL Problem

Figure 6(a) shows our formulation of data placement as an RL problem. We design Sibyl as an RL agent that learns to perform accurate and system-aware data placement decisions by interacting with the hybrid storage system. For each storage request, Sibyl observes multiple workload and system-level features as *state* to make a placement decision. After every *action*, Sibyl receives a *reward* based on the I/O request latency, which reflects both the data placement decision and HSS state. Sibyl aims to learn an optimal data placement policy that improves performance under the current workload and system configuration by minimizing the average request latency. This involves maximizing the hit rate of performance-critical pages in the fast storage device while avoiding the eviction penalty of non-performance-critical pages.

State

At each time step t , the state features for a particular read/write request are collected in an *observation* vector. We perform feature selection to determine the best state features to include in Sibyl's observation vector. Overall, we use six selected features: five features describing the current request and the page it accesses—1) size, 2) type (i.e., read/write), 3) access interval, 4) access count, and 5) current placement, and one feature describing the HSS—the remaining capacity of the fast storage device. We further quantize the state features into a small number of bins to reduce the storage overhead.

Action

At each time step t , in a given state, Sibyl selects an action from all possible actions. In a hybrid storage system with two devices, the possible actions are placing data in 1) the fast storage device or 2) the slow storage device. This is easily extensible to a large number of storage devices (as shown in our 2022 ISCA paper).⁶

Reward

After every data placement decision at time step t (i.e., the arrival of a new storage request), Sibyl gets a reward from the environment at time step $t + 1$, serving as a feedback for the previous action. We craft Sibyl's reward function R as follows:

$$R = \begin{cases} \frac{1}{L_t} & \text{if no page eviction from fast to slow storage} \\ \max(0, \frac{1}{L_t} - R_p) & \text{in case of eviction} \end{cases} \quad (2)$$

where L_t and R_p represent the latency of the last served request and eviction penalty, respectively. If the fast storage is running out of free space, Sibyl evicts non-performance-critical pages to the slow storage. To discourage excessive use of fast storage,

we incorporate an eviction penalty (R_p) into the reward to guide Sibyl to place only performance-critical pages in the fast storage. We empirically set R_p to $0.001 \times L_e$ (L_e is the time required to evict pages from fast to slow storage), which prevents the agent from excessively using the fast storage device.

Sibyl's reward is a function of the request latency because it captures the state of the hybrid storage system as it significantly varies depending on the request type, device type, and the internal state and characteristics of the device (e.g., read/write latencies, garbage collection latency, queuing delays, and error-handling latencies).

Sibyl: Design Overview

We implement Sibyl in the storage management layer of the host system's operating system. As shown in Figure 6(b), Sibyl is composed of two parallel threads: 1) the *RL decision thread*, which performs data placement ④ of the current I/O request while collecting experiences ⑦ (i.e., actions ④ and rewards ⑥) in an *experience buffer* ⑤, and 2) the *RL training thread*, where Sibyl uses the collected *experiences*^b ⑧ to update its decision-making policy online ⑨. Sibyl continuously learns from its past decisions and their impact. Our two-threaded decoupled implementation ensures that policy training does not interfere with latency-critical placement decisions. To enable parallel execution, we use two separate identical neural networks: an inference network ② for making placement decisions, and a training network ⑩ for background learning. Sibyl periodically copies the training network weights to the inference network ⑩ to adapt the policy for current workload and system conditions.

^bExperience is a representation of a transition from one time step to another, in terms of $\langle \text{State}, \text{Action}, \text{Reward}, \text{NextState} \rangle$.

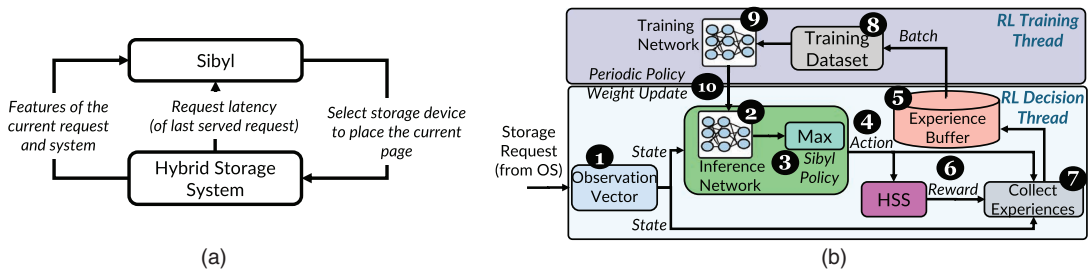


FIGURE 6. (a) Formulation of data placement as an RL problem. (b) Design of Sibyl OS: operating system. Figures adapted from our 2022 ISCA paper.⁶

For every new storage request to the HSS, Sibyl uses the state information ❶ to make a data placement decision ❷. The inference network predicts the Q value for each available action given the state information. Sibyl's policy ❸ selects the action with the maximum Q value and accordingly performs the data placement (and potentially eviction).

Evaluation Methodology

We evaluate Sibyl on a *real system* using two dual-HSS configurations: 1) *performance-oriented HSS: high-end device (H) and middle-end device (M)*, and 2) *cost-oriented HSS: high-end device (H) and low-end device (L)*. The high-, middle-, and low-end devices have a sequential read (write) throughput of 2.4 (2) GB/s, 550 (510) MB/s, and 520 (450) MB/s, respectively. The HSS devices appear to the operating system as one contiguous logical block address space. We implement a lightweight custom block driver to orchestrate I/O requests to the storage devices in the HSS. We develop Sibyl using the TF-Agents framework. We use 14 diverse storage traces that have distinct I/O-access patterns (i.e., randomness and hotness properties) from Microsoft Research Cambridge, collected on real enterprise servers. We compare Sibyl against two heuristic-based (i.e., CDE and history-based page selection (HPS)) and two supervised-learning-based (i.e., Archivist and RNN-HSS) HSS data placement techniques. We compare the aforementioned policies with three boundary scenarios: 1) Slow-Only, where all data reside in the slow storage; 2) Fast-Only, where all data

reside in the fast storage; and 3) Oracle, which exploits complete knowledge of future I/O-access patterns to perform optimal data placement.

Key Results

Figure 7 compares the average request latency of Sibyl against the baseline policies for performance-oriented [Figure 7(a)] and cost-oriented [Figure 7(b)] HSS configurations. All values are normalized to Fast-Only. We observe that Sibyl consistently outperforms all the baselines for all the evaluated workloads in both HSS configurations. In the performance-oriented HSS [Figure 7(a)], where the latency difference between two devices is relatively smaller than that of cost-oriented HSS, Sibyl improves average performance by 28.1%, 23.2%, 36.1%, and 21.6% over CDE, HPS, Archivist, and RNN-HSS, respectively. In the cost-oriented HSS [Figure 7(b)], where there is a large difference between the latencies of the two storage devices, Sibyl improves performance by 19.9%, 45.9%, 68.8%, and 34.1% over CDE, HPS, Archivist, and RNN-HSS, respectively. The larger the latency disparity, the greater the benefits of placing only performance-critical pages in the fast storage to avoid evictions.

We evaluate Sibyl on Tri-HSS with three heterogeneous storage devices. Compared to the state-of-the-art heuristic-based policy, Sibyl improves performance by up to 43.5% on average, which demonstrates that Sibyl's adaptive placement policy generalizes to more devices in an HSS. Extending Sibyl to support additional devices requires only minimal changes to the state and action spaces, highlighting its ease of extensibility.

We conclude that Sibyl's RL-based data placement technique consistently outperforms prior best data placement techniques across diverse workloads and different HSS configurations. Sibyl's workload- and system-aware online learning policy enables it to adapt dynamically to different conditions, making it a generalizable solution for hybrid storage systems. More results are in our 2022 ISCA paper.⁶

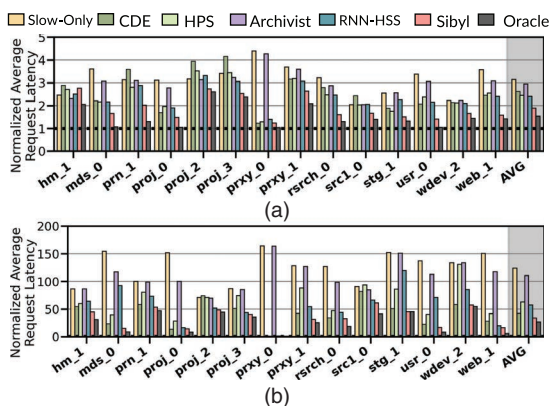


FIGURE 7. Average request latency under two different hybrid storage configurations (normalized to Fast-Only). (a) Performance-oriented HSS. (b) Cost-oriented HSS. Figures adapted from our 2022 ISCA paper.⁶

KEY TAKEAWAYS AND FUTURE OPPORTUNITIES

This article presented our recent efforts on ML-driven intelligent memory system design. We presented three case studies, Pythia, Hermes, and Sibyl, that employ RL and perceptron learning to enable adaptive, data-driven decision making across the multilevel cache hierarchies and hybrid storage systems. Through extensive studies, we showed that these ML-driven policies consistently outperform prior best human-designed approaches with modest overheads, highlighting the

practicality of online learning in real systems. These case studies also reveal broader opportunities, and challenges, in how such mechanisms are *integrated* and *extended* across the memory hierarchy.

Better Integration and Coordination of ML-Driven Mechanisms

Although Pythia, Hermes, and Sibyl improve performance independently and in combination (e.g., Hermes with Pythia), their interactions expose additional opportunities for deeper system-level integration. For example, although integrating Hermes with Pythia improves performance on average, we observe that a naïve integration often fails to fully realize their combined performance potential. This motivates designing holistic coordination mechanisms—potentially ML-driven—that jointly coordinate prefetching and off-chip prediction mechanisms. Recently, we proposed Athena,¹¹ which exploits online RL to coordinate off-chip prediction with multiple prefetchers employed throughout the cache hierarchy.

Extending ML-Driven Design Principles Across the Memory Hierarchy

The ML-driven design principles presented in this article can be broadly extended for many data-driven decision-making processes across the memory hierarchy, including 1) co-optimization of caching, prefetching, and memory scheduling mechanisms; 2) co-optimization of thread and memory scheduling decisions; 3) data placement and migration in disaggregated memory systems; and 4) coordinated data placement and migration in hybrid storage systems. These problems exhibit complex, nonlinear interactions between workload behavior and system state, making them well suited for lightweight, online ML approaches.

Collectively, these directions highlight the potential of ML-driven approaches to enable adaptive, self-optimizing memory systems that deliver performance and efficiency gains beyond traditional designs, and we hope that this article inspires further research in intelligent ML-driven memory system design.

REFERENCES

1. G. F. Oliveira, J. Gómez-Luna, S. Ghose, A. Boroumand, and O. Mutlu, "Accelerating neural network inference with processing-in-DRAM: From the edge to the cloud," *IEEE Micro*, vol. 42, no. 6, pp. 25–38, Nov./Dec. 2022, doi: [10.1109/MM.2022.3202350](https://doi.org/10.1109/MM.2022.3202350).
2. G. Singh et al., "FPGA-based near-memory acceleration of modern data-intensive applications," *IEEE Micro*, vol. 41, no. 4, pp. 39–48, Jul./Aug. 2021, doi: [10.1109/MM.2021.3088396](https://doi.org/10.1109/MM.2021.3088396).
3. M. Alser et al., "Accelerating genome analysis: A primer on an ongoing journey," *IEEE Micro*, vol. 40, no. 5, pp. 65–75, Sep./Oct. 2020, doi: [10.1109/MM.2020.3013728](https://doi.org/10.1109/MM.2020.3013728).
4. O. Mutlu, "Intelligent architectures for intelligent computing systems," in *Proc. Des., Autom. Test Europe Conf. Exhib. (DATE)*, 2021, pp. 318–323, doi: [10.23919/DATE51398.2021.9474073](https://doi.org/10.23919/DATE51398.2021.9474073).
5. N. P. Jouppi, "Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers," in *Proc. 17th Annu. Int. Symp. Comput. Archit.*, 1990, pp. 364–373, doi: [10.1145/325164.325162](https://doi.org/10.1145/325164.325162).
6. G. Singh et al., "Sibyl: Adaptive and extensible data placement in hybrid storage systems using online reinforcement learning," in *Proc. 49th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2022, pp. 320–336.
7. R. Bera et al., "Pythia: A customizable hardware framework using online reinforcement learning," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2021, pp. 1121–1137.
8. R. Bera et al., "Hermes: Accelerating long-latency load requests via perceptron-based off-chip load prediction," in *Proc. 55th IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2022, pp. 1–18, doi: [10.1109/MICRO56248.2022.00015](https://doi.org/10.1109/MICRO56248.2022.00015).
9. A. Boroumand et al., "Google workloads for consumer devices: Mitigating data movement bottlenecks," in *Proc. 23rd Int. Conf. Architectural Support Program. Lang. Operating Syst. (ASPLOS)*, 2018, pp. 316–331.
10. A. Boroumand et al., "Google neural network models for edge devices: Analyzing and mitigating machine learning inference bottlenecks," in *Proc. 30th Int. Conf. Parallel Archit. Compilation Techn. (PACT)*, 2021, pp. 159–172, doi: [10.1109/PACT52795.2021.00019](https://doi.org/10.1109/PACT52795.2021.00019).
11. R. Bera et al., "Athena: Synergizing data prefetching with off-chip prediction via online reinforcement learning," 2026. [Online]. Available: https://people.inf.ethz.ch/omutlu/pub/Athena_hpca26-talk.pdf
12. O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungrun, "A modern primer on processing in memory," in *Proc. Emerg. Comput.: From Devices Syst.*, 2022, pp. 171–243.
13. O. Mutlu, A. Olgun, and İ. E. Yüksel, "Memory-centric computing: Solving computing's memory problem," in *Proc. IEEE Int. Memory Workshop (IMW)*, 2025, pp. 1–4, doi: [10.1109/IMW61990.2025.11026935](https://doi.org/10.1109/IMW61990.2025.11026935).

14. O. Mutlu, A. Olgun, G. F. Oliveira, and I. E. Yuksel, "Memory-centric computing: Recent advances in processing-in-DRAM (invited)," in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, 2024, pp. 1–4, doi: [10.1109/IEDM50854.2024.10873410](https://doi.org/10.1109/IEDM50854.2024.10873410).

RAHUL BERA is a senior researcher at, ETH Zürich, Zürich, 8092, Switzerland. Bera received his Ph.D. degree in electrical engineering. Contact him at rahul.bera@safari.ethz.ch.

RAKESH NADIG is a Ph.D. student in electrical engineering at the SAFARI Research Group, ETH Zürich, Zürich, 8092, Switzerland. His current research interest lies in storage system design and near-data processing. Nadig received his master's degree in electrical and computer engineering from

the University of California, Irvine. Contact him at rakesh.nadig@safari.ethz.ch.

ONUR MUTLU is a full professor at ETH Zürich, Zürich, 8092, Switzerland, and leads the SAFARI Research Group. He previously held the William D. and Nancy W. Strecker Early Career Professorship at Carnegie Mellon University. His research interests include computer architecture, computing systems, hardware security, memory and storage systems, and bioinformatics, with a major focus on designing fundamentally energy-efficient, high-performance, and robust computing systems. Mutlu received his Ph.D. and M.S. degrees in electrical and computer engineering from the University of Texas at Austin. He is a Fellow of IEEE and the Association for Computing Machinery. Contact him at onur.mutlu@safari.ethz.ch.



RECOGNIZE EXCELLENCE

COMPUTER SCIENCE & ENGINEERING AWARDS

IEEE Computer Society honors groundbreaking achievements in research and contributions to the global computing community.

AWARDS INCLUDE:

- Technical Achievements
- Teaching & Education
- Student Distinction & Scholarships
- Exemplary Service



To learn more, visit
computer.org/awards



**IEEE
COMPUTER
SOCIETY**